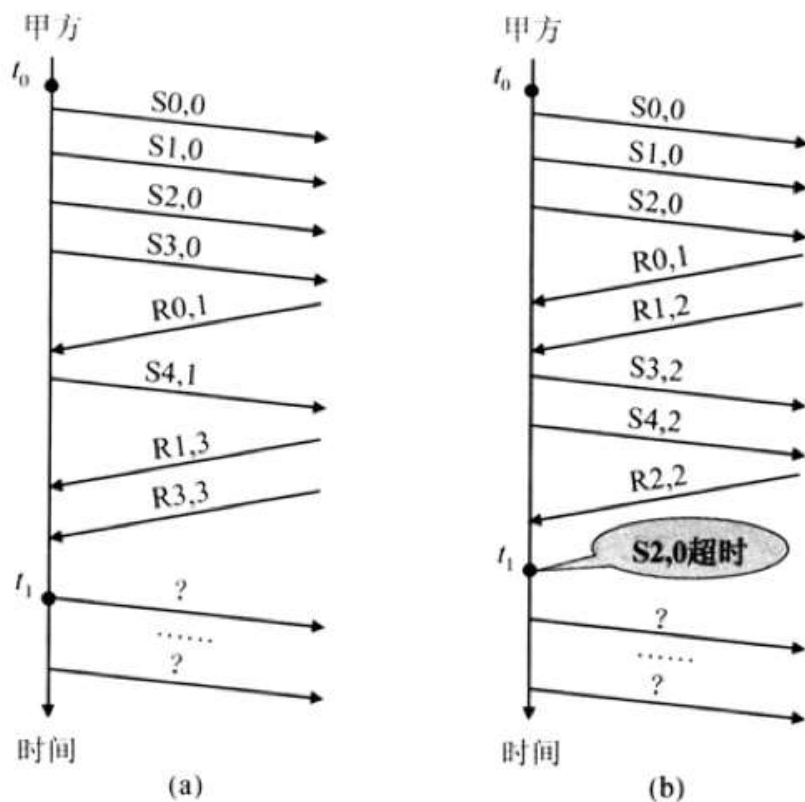




- (1) 对于图(a), t_0 时刻到 t_1 时刻期间, 甲方可以断定乙方已正确接收的数据帧数是多少? 正确接收的是哪几个帧 (请用 $S_{x,y}$ 形式给出)?
- (2) 对于图(a), 从 t_1 时刻起, 甲方在不出现超时且未收到乙方新的数据帧之前, 最多还可以发送多少个数据帧? 其中第一个帧和最后一个帧分别是哪个 (请用 $S_{x,y}$ 形式给出)?
- (3) 对于图(b), 从 t_1 时刻起, 甲方在不出现新的超时且未收到乙方新的数据帧之前, 需要重发多少个数据帧? 重发的第一个帧是哪个 (请用 $S_{x,y}$ 形式给出)?
- (4) 甲方可以达到的最大信道利用率是多少?

2017 年计算机学科专业基础参考答案

一、单项选择题

1. 【解析】B。sum+=++i; 相当于 ++i; sum=sum+i;。进行到第 k 趟循环, $sum=(1+k)*k/2$ 。显然需要进行 $O(n^{1/2})$ 趟循环, 因此这也是该函数的时间复杂度。递归数列求和, sum 是 n^2 级别, 所以复杂度为 $O(n^{1/2})$ 。
2. 【解析】C。采用非递归方式重写递归程序时不一定使用栈; 只要确定了入栈次序, 不确定出栈次序栈; 是一种受限的线性表, 不允许在其两端进行操作。I、III、IV 错误。I 的反例: 计算斐波拉契数列迭代实现只需要一个循环即可实现。III 的反例: 入栈序列为 1、2, 进行如下操作 PUSH、PUSH、POP、POP, 出栈次序为 2、1; 进行如下操作 PUSH、POP、PUSH、POP, 出栈次序为 1、2。IV, 栈是一种受限的线性表, 只允许在一端进行操作。因此 II 正确。
3. 【解析】A。三元组表的结点存储了行 row、列 col、值 value 三种信息, 是主要用来存储

稀疏矩阵的一种数据结构。十字链表将行单链表和列单链表结合起来存储稀疏矩阵。邻接矩阵空间复杂度达 $O(n^2)$, 不适于存储稀疏矩阵。二叉链表又名左孩子右兄弟表示法, 可用于表示树或森林。因此 A 正确。

4.【解析】B。先序序列是先父结点, 接着左子树, 然后右子树。中序序列是先左子树, 接着父结点, 然后右子树, 递归进行。如果所有非叶结点只有右子树, 先序序列和中序序列都是先父结点, 然后右子树, 递归进行, 因此 B 正确。

5.【解析】B。后序序列是先左子树, 接着右子树, 最后父结点, 递归进行。根结点左子树的叶结点首先被访问, 它是 e。接下来是它的父结点 a, 然后是 a 的父结点 c。接着访问根结点的右子树。它的叶结点 b 首先被访问, 然后是 b 的父结点 d, 再者是 d 的父结点 g。最后是根结点 f。因此 d 与 a 同层, B 正确。

6.【解析】D。哈夫曼编码是前缀编码, 各个编码的前缀各不相同, 因此直接拿编码序列与哈夫曼编码一一比对即可。序列可分割为 0100 011 001 001 011 11 0101, 译码结果是 afefgd, 选项 D 正确。

7.【解析】B。 $N_0+N_1+N_2+N_3+N_4=16+1, 4*N_4+3*N_3+x*N_2+y*N_1+1=N_0+N_1+N_2+N_3+N_4, N_4=3, N_3=4$, 代入, 顶点个数至少为 11。

无向图边数的两倍等于各顶点度数的总和。由于其他顶点的度均小于 3, 可以设它们的度都为 2, 设它们的数量是 X, 可列出方程 $4*3+3*4+2*x=16*2$, 解得 $x=3$ 。 $4+3+3=11$, B 正确。

8.【解析】A。折半查找判定树实际上是一棵二叉排序树, 它的中序序列是一个有序序列。可以在树结点上依次填上相应的元素, 符合折半查找规则的树即是所求。B 选项 4、5 相加除 2 向上取整, 7、8 相加除 2 向下取整, 矛盾。C 选项, 3、4 相加除 2 向上取整, 6、7 相加除 2 向下取整, 矛盾。D 选项, 1、10 相加除 2 向下取整, 6、7 相加除 2 向上取整, 矛盾。A 符合折半查找规则, 因此正确。

9.【解析】B。B+树是应文件系统所需而产生的 B-树的变形, 前者比后者更加适用于实际应用中的操作系统的文件索引和数据库索引, 因为前者磁盘读写代价更低, 查询效率更加稳定。编译器中的词法分析使用有穷自动机和语法树。网络中的路由表快速查找主要靠高速缓存、路由表压缩技术和快速查找算法。系统一般使用空闲空间链表管理磁盘空闲块。所以 B 正确。

10.【解析】B。归并排序的时间复杂度更低, 运行效率更高。归并排序代码比选择插入排序更复杂, 前者空间复杂度是 $O(n)$, 后者是 $O(1)$ 。但是前者时间复杂度是 $O(n\log n)$, 后者是 $O(n^2)$ 。所以 B 正确。

11.【解析】D。希尔排序需要多次分割链表进行直接插入排序, 堆排序需要多次调整堆。这些操作都不利于在链表上进行。插入排序、选择排序、起泡排序原本时间复杂度是 $O(n^2)$, 更换为链式存储后的时间复杂度还是 $O(n^2)$ 。希尔排序和堆排序都利用了顺序存储的随机访问特性, 而链式存储不支持这种性质, 所以时间复杂度会增加, 因此选 D。

12.【解析】C。运行时间=指令数*CPI/主频。M1 的时间=指令数*2/1.5, M2 的时间=指令数*1/1.2。两者之比为 $(2/1.5):(1/1.2)=1.6$ 。

13.【解析】C。由四个 DRAM 芯片采用交叉编址方式构成主存可知主存地址最低二位表示该字节存储的芯片编号。Double 型变量占 64 位, 8 个字节。他的主存地址 804001AH 最低

二位是 10 说明他从编号为 2 的芯片开始存储, 编号从 0 开始。一个存储周期可以对所有芯片各读取一个字节, 因此需要三轮, 选 C。

14. 【解析】A。时间局部性是一旦一条指令执行了, 则在不久的将来他可能再次被执行。空间局部性是一旦一个存储单元被访问, 那么他附近的存储单元也很快被访问。显然, 这里的循环指令本身具有时间局部性, 他对数组 a 的访问具有空间局部性。

15. 【解析】D。在变址操作时, 计算机指令中的地址与变址寄存器中地址相加, 得到有效地址, 指令提供数组的首地址, 由变址寄存器来定位数据中的各元素。所以他最适合按下标顺序访问一维数组元素, 选 D。

16. 【解析】A。三地址指令有 29 条, 所以他的操作码至少为 5 位, 以五位计算, 他剩余 3 种操作码给二地址。而二地址另外多了 6 位给操作码, 因此数量最大达 $3 \times 64 = 192$, 所以指令字长最少为 23 位, 因此计算机按字节编址, 需要是 8 的倍数, 所以指令字长至少应该是 24 位。

17. 【解析】C。超标量是指在 CPU 中有一条以上的流水线, 并且每个时钟周期内可以完成一条以上的指令, 其实质是以空间换时间。I 错误, 他不影响流水线功能段的处理时间。II, III 正确。

18. 【解析】B。主存存储指令和数据, 在 CPU 外, 由 RAM 和 ROM 实现。控制存储器用来存放实现指令系统的所有微指令是一种只读型存储器, 机器运行时, 只读不写, 在 CPU 控制器内。CS 按照微指令的地址访问。

19. 【解析】A。五段流水线可分为取值 IF, 译码/取数 ID, 执行 EXC, 存储器读 MEM, 写回 WB。数字系统中, 各个子系统通过数据总线连接形成的数据传送路径称为数据通路, 包括程序计数器, 算术逻辑运算部件, 通用寄存器组, 取指部件等, 不包括控制部件。

20. 【解析】D。多总线结构用速率高的总线连接高速设备, 用速率低的总线连接低速设备。一般来说 CPU 是计算机的核心, 是计算机中速度最快的设备之一。A 正确。突发传输把多个数据单元作为一个独立传输处理, 从而最大化设备的吞吐量, 可提高读写效率, B 正确。各总线通过桥接器相连, 起流量交换的作用。PCI-Express $\times$ 16 采用串行传输方式。

21. 【解析】D。IO 端口又称为 IO 接口, 是 CPU 与设备中间的交界面。由于主机和 IO 设备的工作方式和工作速度有很大的差异, IO 端口应运而生。执行一条指令时, CPU 使用地址总线选择所请求的 IO 端口, 使用数据总线在 CPU 寄存器和端口之间传输数据。

22. 【解析】B。多重中断系统在保护被中断进程现场时关中断, 执行中断处理程序时开中断。B 错误。CPU 一般在一条指令执行结束后采样中断请求信号, 查看是否有中断请求, 然后决定是否响应中断。AD 正确。中断请求一般来自 CPU 以外的事件, 异常一般发生在 CPU 内部。中断处理期间 CPO 处于开中断状态, 以便接收高优先级的中断。

23. 【解析】D。根据 FIFS 和短作业优先调度算法计算即可。先来先服务调度算法是作业来得越早, 优先级越高, 因此会选择 J1。短作业优先调度算法是作业运行时间越短, 优先级越高, 因此会选择 J3。所以 D 正确。

24. 【解析】C。执行系统调用的过程是这样的: 正在运行的进程先传递系统调用参数, 然后由陷入(trap)指令负责将用户态转化为内核态, 并将返回地址压入堆栈以备后用, 接下来 CPU 执行相应的内核态服务程序, 最后返回用户态。所以 C 正确。

25.【解析】B。根据最佳适应算法计算即可。回收起始地址为 60K、大小为 140KB 的分区时，它与表中第一个分区和第四个分区合并，成为起始地址为 20K、大小为 380KB 的分区，剩余 3 个空闲分区。在回收内存后，算法会对空闲分区链接分区大小由小到大进行排序，表中的第二个分区排第一。所以选择 B。

26.【解析】D。数据存储于硬盘中时以簇为基本单位，故选 D。

27.【解析】B。当前进程的时间片用完后，该进程状态由执行态变为就绪态。进程切换带来系统开销，切换次数越多，开销越大，A 正确。当前进程的时间片用完后，它的状态由执行态变为就绪态，B 错误。时钟中断是系统中特定的周期性时钟节拍。操作系统通过它来确定时间间隔，实现时间的延时和任务的超时，C 正确。现代操作系统为了保证性能最优，通常根据响应时间、系统开销、进程数量、进程运行时间、进程切换开销等因素确定时间片大小，D 正确。

28.【解析】D。多道程序系统通过组织作业（编码或数据）使 CPU 总有一个作业可执行，从而提高了 CPU 的利用率、系统吞吐量和 I/O 设备利用率，I、III、IV 是优点。但系统要付出额外的开销来组织作业和切换作业，II 错误。所以选 D。

29.【解析】B。磁盘逻辑格式化程序所做的工作是建立文件系统的根目录、对保存空闲磁盘块信息的数据结构进行初始化。一个新的磁盘是一个空白版，必须分成扇区以便磁盘控制器能读和写，这个过程称为低级格式化（或物理格式化）。低级格式化为磁盘的每个扇区采用特别的数据结构，包括校验码，III 错误。为了使用磁盘存储文件，操作系统还需要将其数据结构记录在磁盘上。这分为两步，第一步是将磁盘分为由一个或多个柱面组成的分区，每个分区可以作为一个独立的磁盘，I 错误。在分区之后，第二步是逻辑格式化（创建文件系统）。在这一步，操作系统将初始的文件系统数据结构存储到磁盘上。这些数据结构包括空闲和已分配的空间和一个初始为空的目录，II、IV 正确。所以选 B。

30.【解析】D。可以把用户访问权限抽象为一个矩阵，行代表用户，列代表访问权限。这个矩阵有 4 行 5 列，1 代表 true，0 代表 false，所以需要 20 位，选 D。

31.【解析】B。硬链接指通过索引结点进行连接。一个文件在物理存储器上有一个索引结点号。存在多个文件名指向同一个索引结点，II 正确。两个进程各自维护自己的文件描述符，III 正确，I 错误。所以选择 B。

32.【解析】B。系统将数据从磁盘读到内存的过程为：初始化 DMA 控制器并启动磁盘、从磁盘传输一块数据到内存缓冲区、DMA 控制器发出中断请求、执行“DMA 结束”中断服务程序。

33.【解析】A。OSI 参考模型共有 7 层，从上到下为：应用层、表示层、会话层、传输层、网络层、数据链路层、物理层。除物理层和应用层之外，其他各层在封装 PDU 时均引入 20B 的额外开销，则说明表示层、会话层、传输层、网络层、数据链路层都会添加 20B 的数据，所以共添加 $20 \times 5 = 100\text{B}$ 的数据，应用层共有 500B 数据，400B 原始数据，所以则应用层数据传输效率约为 $400/500 = 80\%$ 。

34.【解析】D。可用奈奎斯特采样定理计算无噪声情况下的极限数据传输速率，用香农第二定理计算有噪声信道极限数据传输速率。 $2W \log_2 N \geq W \log_2(1 + S/N)$ 。W 是信道带宽，N 是

信号状态数, S/N 是信噪比, 将数据代入计算可得 $N \geq 32$, 选 D。分贝数 $= 10 \log_{10} S/N$ 。

35. 【解析】B。地址 1 指的是接收端的 mac 地址, 地址 2 指的是传输帧的介质的 mac 地址, 地址 3 指的是 subnet 连接路由器的端口地址, 地址 4 用于 AP 自组织模式下相互转发。本题里只用到地址 1~3。帧传送过程是这样的: 从主机 H 到路由器 R, 分为两个阶段。第一个阶段从 H 到 AP, 用 IEEE802.11 协议, 第二个阶段从 AP 到 R, 用 IEEE802.3 协议 (以太网帧)。第一个阶段, AP 作为接收端, 将地址 1 填充为自己的 mac 地址。无线网络是传输介质, AP 将地址 2 填充为无线网络的 mac 地址, 此处无线网络的 mac 地址就是主机的 mac 地址。路由器 R 对应的端口作为子网连接的端口, AP 将对应的端口 mac 地址填充到地址 3。第二个阶段, 目的地是路由器 R, 以太网帧的目的 mac 地址就是 R 的 mac 地址。帧由 AP 发出, 源 mac 地址就是 AP 的 mac 地址。

36. 【解析】A。0.0.0.0 不能作为目的 IP 地址。根据 RFC 文档描述, 0.0.0.0/32 可以作为本主机在本网络上的源地址。127.0.0.1 是回送地址, 以它为目的 IP 地址的数据将被立即返回到本机。200.10.10.3 是 C 类 IP 地址。255.255.255.255 是广播地址。

37. 【解析】D。RIP 协议使用 UDP 广播, OSPF 使用 IP 协议, BGP 使用 TCP 协议。RIP 是一种分布式的基于距离向量的路由选择协议, 通过广播 UDP 报文来交换路由信息。OSPF 是一个内部网关协议, 不使用传输协议, 如 UDP 或 TCP, 而是直接用 IP 封装它的数据。BGP 是一个外部网关协议, 用 TCP 封装它的数据。因此选 D。

38. 【解析】C。子网号为 7 位, 子网可分配的最大 IP 地址个数为 $2^{16-7}-2=510$ 。这个网络有 16 位的主机号, 平均分成 128 个规模相同的子网, 每个子网有 7 位的子网号, 9 位的主机号。除去一个网络地址和广播地址, 可分配的最大 IP 地址个数是 $29-2=512-2=510$, 故选 C。

39. 【解析】A。按照慢开始算法, 发送窗口 $= \min\{\text{拥塞窗口}, \text{接收窗口}\}$, 初始的拥塞窗口为最大报文段长度 1KB。每经过一个 RTT, 拥塞窗口翻倍, 因此需至少经过 5 个 RTT, 发送窗口才能达到 32KB, 所以选 A。这里假定乙能及时接收到的数据, 空闲的接收缓存 $\geq 32\text{KB}$ 。

40. 【解析】C。数据连接的端口号由用户指定, 不一定为 TCP20 端口 FTP 协议使用控制连接和数据连接, 控制连接存在于整个 FTP 会话过程中, 数据连接在每次文件传输时才建立, 传输结束就关闭, A 和 B 是正确的。默认情况下 FTP 协议使用 TCP20 端口进行数据连接, TCP21 端口进行控制连接。但是是否使用 TCP20 端口建立数据连接与传输模式有关, 主动方式使用 TCP20 端口, 被动方式由服务器和客户端自行协商决定, C 错, D 对。所以选 C。

41. 【解析】

(1) 算法的基本设计思想

表达式树的中序序列加上必要的括号即为等价的中缀表达式。可以基于二叉树的中序遍历策略得到所需的表达式。

表达式树中分支结点所对应的子表达式的计算次序, 由该分支结点所处的位置决定。为得到正确的中缀表达式, 需要在生成遍历序列的同时, 在适当位置增加必要的括号。显然, 表达式的最外层 (对应根结点) 及操作数 (对应叶结点) 不需要添加括号。

(2) 算法实现

```
void BtreeToE(BTree *root){
```

```

    BtreeToExp(root, 1); //根的高度为 1
}
void BtreeToExp(BTree * root, int deep){
    if( root == NULL) return;
    else if(root->left == NULL && root->right == NULL)//若为叶结点
        printf("%s", root->data); //输出操作数
    else{
        if(deep>1)
            printf("("); //若有子表达式则加 1 层括号
        BtreeToExp(root->left, deep+1);
        printf("%s", root->data); //输出操作符
        BtreeToExp(root->right, deep+1);
        if(deep>1)
            printf(")"); //若有子表达式则加 1 层括号
    }
}
}

```

42. 【解析】

(1) Prim 算法属于贪心策略。算法从一个任意的顶点开始，一直长大到覆盖图中所有顶点为止。算法每一步在连接树集合 S 中顶点和其他顶点的边中，选择一条使得树的总权重增加最小的边加入集合 S 。当算法终止时， S 就是最小生成树。

- ① S 中顶点为 A ，候选边为 (A, D) ， (A, B) ， (A, E) ，选择 (A, D) 加入 S 。
- ② S 中顶点为 A, D ，候选边为 (A, B) ， (A, E) ， (D, E) ， (C, D) ，选择 (D, E) ，加入 S 。
- ③ S 中顶点为 A, D, E ，候选边为 (A, B) ， (C, D) ， (C, E) ，选择 (C, E) 加入 S 。
- ④ S 中顶点为 A, D, E, C ，候选边为 (A, B) ， (B, C) ，选择 (B, C) 加入 S 。
- ⑤ S 就是最小生成树。

依次选出的边为 (A, D) ， (D, E) ， (C, E) ， (B, C) 。

(2) 图 G 的 MST 是唯一的。

(3) 当带权连通图的任意一个环中所包含的边的权值均不相同时，其 MST 是唯一的。

43. 【解析】

(1) 由于 i 和 n 是 unsigned 型，故 “ $i \leq n-1$ ” 是无符号数比较， $n=0$ 时， $n-1$ 的机器数为全 1，值是 $2^{32}-1$ ，为 unsigned 型可表示的最大数，条件 “ $i \leq n-1$ ” 永真，因此出现死循环。

若 i 和 n 改为 int 类型，则不会出现死循环。

因为 “ $i \leq n-1$ ” 是带符号整数比较， $n=0$ 时， $n-1$ 的值是 -1，当 $i=0$ 时条件 “ $i \leq n-1$ ” 不成立，此时退出 for 循环。

(2) $f1(23)$ 与 $f2(23)$ 的返回值相等。 $f1(23)$ 的机器数是 00FF FFFFH， $f2(23)$ 的机器数是 4B7F FFFFH。

(3) 当 $n=24$ 时， $f(24)=1\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ B$ ，而 float 型数只有 24 位有效位，

舍入后数值增大，所以 $f2(24)$ 比 $f1(24)$ 大 1。

(4) 显然 $f(31)$ 已超出了 `int` 型数据的表示范围，用 $f1(31)$ 实现时得到的机器数为 32 个 1，作为 `int` 型数解释时其值为 -1，即 $f1(31)$ 的返回值为 -1。

因为 `int` 型最大可表示数是 0 后面加 31 个 1，故使 $f1(n)$ 的返回值与 $f(n)$ 相等的最大 n 值是 30。

(5) IEEE754 标准用“阶码全 1、尾数全 0”表示无穷大。 $f2$ 返回值为 `float` 型，机器数 7F80 0000H 对应的值是 $+\infty$ 。

当 $n=126$ 时， $f(126)=2^{127}-1=1.1\cdots1\times2^{126}$ ，对应阶码为 $127+126=253$ ，尾数部分舍入后阶码加 1，最终阶码为 254，是 IEEE754 单精度格式表示的最大阶码。故使 $f2$ 结果不溢出的最大 n 值为 126。

当 $n=23$ 时， $f(23)$ 为 24 位 1，`float` 型数有 24 位有效位，所以不需舍入，结果精确。故使 $f2$ 获得精确结果的最大 n 值为 23。

44. 【解析】

(1) M 为 CISC。

M 的指令长短不一，不符合 RISC 指令系统特点。

(2) $f1$ 的机器代码占 96B。

因为 $f1$ 的第一条指令“`push ebp`”所在的虚拟地址为 0040 1020H，最后一条指令“`ret`”所在的虚拟地址为 0040 107FH，所以， $f1$ 的机器指令代码长度为 $0040\ 107FH-0040\ 1020H+1=60H=96$ 个字节。

(3) $CF=1$ 。

`cmp` 指令实现 i 与 $n-1$ 的比较功能，进行的是减法运算。在执行 $f1(0)$ 过程中， $n=0$ ，当 $i=0$ 时， $i=0000\ 0000H$ ，并且 $n-1=FFFF\ FFFFH$ 。因此，当执行第 20 条指令时，在补码加/减运算器中执行“0 减 $FFFF\ FFFFH$ ”的操作，即 $0000\ 0000H+0000\ 0000H+1=0000\ 0001H$ ，此时，进位输出 $C=0$ ，减法运算时的借位标志 $CF=C\oplus 1=1$ 。

(4) $f2$ 中不能用 `shl` 指令实现 $power*2$ 。

因为 `shl` 指令用来将一个整数的所有有效数位作为一个整体左移；而 $f2$ 中的变量 `power` 是 `float` 型，其机器数中不包含最高有效数位，但包含了阶码部分，将其作为一个整体左移时并不能实现“乘 2”的功能，因而 $f2$ 中不能用 `shl` 指令实现 $power*2$ 。

45. 【解析】

(1) 函数 $f1$ 的代码段中所有指令的虚拟地址的高 20 位相同，因此 $f1$ 的机器指令代码在同一页中，仅占用 1 页。

(2) `push ebp` 指令的虚拟地址的最高 10 位（页目录号）为 00 00000001，中间 10 位（页表索引）为 00 0000 0001，所以，取该指令时访问了页目录的第 1 个表项，在对应的页表中访问了第 1 个表项。

(3) 在执行 `scanf()` 的过程中，进程 P 因等待输入而从执行态变为阻塞态。输入结束时，P 被中断处理程序唤醒，变为就绪态。P 被调度程序调度，变为运行态。CPU 状态会从用户态变为内核态。

46. 【解析】

semaphore mutex_y1=1; //mutex_y1 用于 thread1 与 thread3 对变量 y 的互斥访问;

semaphore mutex_y2=1; //mutex_y2 用于 thread2 与 thread3 对变量 y 的互斥访问;

semaphore mutex_z=1; //mutex_z 用于变量 z 的互斥访问;

互斥代码如下:

```
thread1 {  
    cnum w;  
    wait(mutex_y1);  
    w=add(x, y);  
    signal(mutex_y1);  
    .....  
}
```

```
thread2 {  
    cnum w;  
    wait(mutex_y2);  
    wait(mutex_z);  
    w=add(y, z);  
    signal(mutex_z);  
    signal(mutex_y2);  
    .....  
}
```

```
thread3 {  
    cnum w;  
    w.a=1;  
    w.b=1;  
    wait(mutex_z);  
    z=add(z, w);  
    signal(mutex_z);  
    wait(mutex_y1);  
    wait(mutex_y2);  
    y=add(y, w);  
    signal(mutex_y1);  
    signal(mutex_y2);  
    .....  
}
```


47. 【解析】

(1) t_0 时刻到 t_1 时刻期间, 甲方可以断定乙方已正确接收了 3 个数据帧, 分别是 $S_{0,0}$, $S_{1,0}$, $S_{2,0}$ 。 $R_{3,3}$ 说明乙发送的数据帧确认号是 3, 即希望甲发送序号 3 的数据帧, 说明乙已经接收了序号为 0~2 的数据帧。

(2) 从 t_1 时刻起, 甲方最多还可以发送 5 个数据帧, 其中第一个帧是 $S_{3,2}$ (1 分), 最后一个数据帧是 $S_{7,2}$ (1 分)。发送序号 3 位, 有 8 个序号。在 GBN 协议中, 序号个数 $\geq$ 发送窗口+1, 所以这里发送窗口最大为 7。此时已发送了 $S_{3,0}$ 和 $S_{4,1}$, 所以最多还可以发送 5 个帧。

(3) 甲方需要重发 3 个数据帧 (1 分), 重发的第一个帧是 $S_{2,3}$ (1 分)。在 GBN 协议中, 接收方发送了 N 帧后, 检测出错, 则需要发送出错帧及其之后的帧。 $S_{2,0}$ 超时, 所以重发的第一帧是 S_2 已收到乙的 R_2 帧, 所以确认号应为 3。

(4) 甲方可以达到的最大信道利用率是

$$\frac{7 \times \frac{8 \times 1000}{100 \times 10^6}}{0.96 \times 10^{-3} + 2 \times \frac{8 \times 1000}{100 \times 10^6}} \times 100\% = 50\%$$